6

National University

of Computer & Emerging Sciences Peshawar Campus

Student Name: Muhammad Bilal Section: ~~B~~ A Roll No: P15-6068

Program: CS 15
Semester: FALL – 2016
Time Allowed: 1 hour
Course: Data Structure

No Answer Sheet Required

Examination: SESSIONAL – II
Total Marks: 30
Date: 28th November, 2016
Instructor: Mr. Fazl-e-Basit

14.5

~~24~~

NOTE: ***NO ANSWER SHEET REQUIRED.*** Attempt all questions.

Q. 1) Assume Doubly Ring (**DR**) filled with unique integers, exists in memory. User gives a key value to search it in DR. Write a C/C++ **recursive** code that tells whether the key is searchable quickly by using next pointers or by using previous pointers in DR. Assume head of DR is passed initially to recursive function. Zero credit for Iterative solution. [8]

1

```
void search (node * head)
{
    temp = head
    ~~if (head->~~
    int a = count next (temp, int key);
    ~~ab b = count~~
    ~~while~~ ~~while (temp->nxt~~
    while (temp->nat != NULL)
    {
        a. ~~count~~ pre
        temp = temp->next;
    }
        temp2 = temp;
    ~~int count pre (node *~~
    int b = count pre (temp2)
    if (a > b)
    { cout<< "from next"
    } else { "from right" }
}
```

2

```
int countnext (node *~~head~~ temp, key)
{
    if ~~(head->next)~~ == temp
    { ~~retur~~
    if (temp->next == head)
    { return k countn;
    }
    if (temp->data == key)
    { return countn;
    }
    else
    { countn++;
    return countnext (temp->next);
    }
}
```

3

```
int count pre (node *temp, int key)
    if (temp->next == head)
    {
        return count L;
    }
    if (temp->data == key)
    { return countL;
    }
    else return count (temp->pre);
}
```

Q. 2) Write a code for **Enqueue( )** and **Dequeue( )** operations using **Array** data structure. Code should have all necessary checks for overflow and underflow in both the functions. [8]

```
#include <iostream>
using namespace std;
class Array-Queue
{   int *Que ;   int *rare;
    int *front;   int max;
     int current;   int *t;

 Public:
   Array-Que()
 {  cout<<"Enter Queue size";
      cin>> max;
      Que= new int[max]
     front=rare=t=NULL;
       current=0; }

void enqueue()
{    if(front==NULL)
      {  front = Que;
         ~~cin>>*front~~;
         rare = Que;
         cin>>*rare;
         *front = *rare;
         ~~[illegible]~~ current++;
         return;
      }
 if(current==max)
   { cout<<"Overflow";
              return;
   }
else{  rare++;
       cin>>*rare; t=rare;
       current++;
       return;
    }
 if(current==0)
  { cout<<"under flow"
     return;
  }

}

void deque ( )
{   if(front==rare)
     { cout<<*front<<endl;
       front++; t++; current--;
     }
   if (front==t) 
     { cout<<"~~under~~overflow";
       return;
     }
   if(front!=NULL)
     { cout<<*front<<endl;
        front++; current--;
     }
   if (front==NULL)
    { cout<<"Underflow";
    }
}
```

4

1

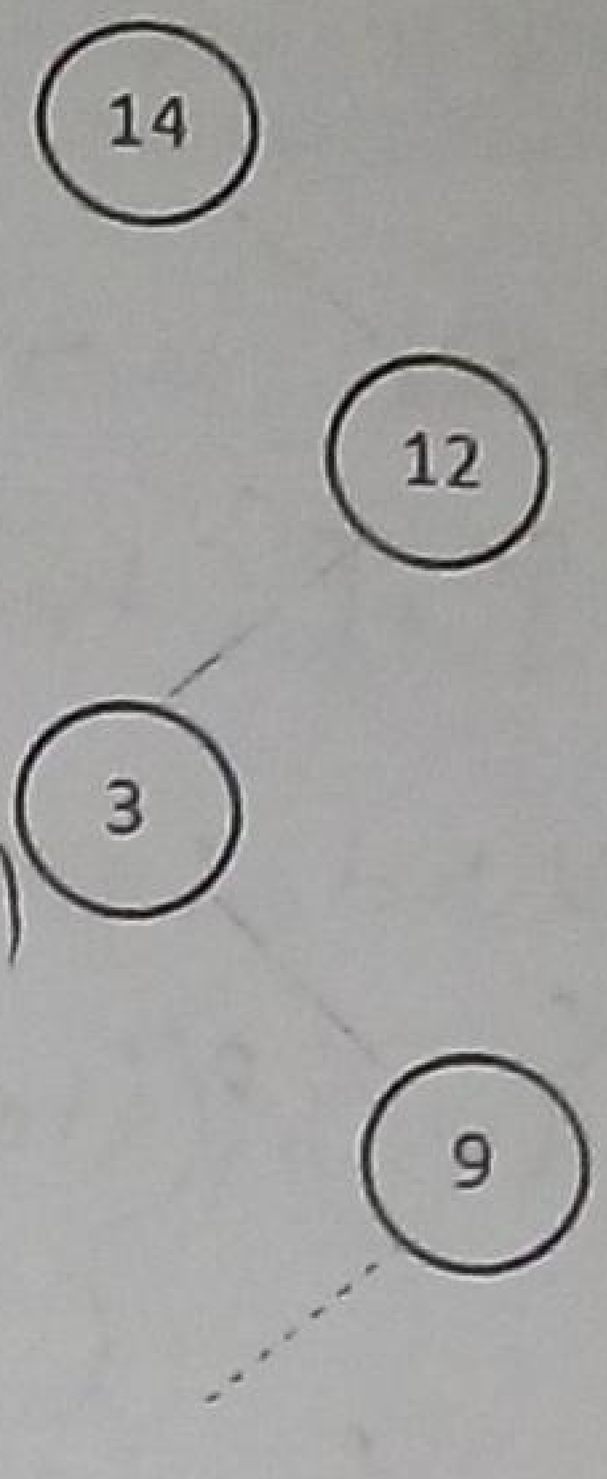
1

1

0.5

Zig-zag

Q. 3) Write a **Recursive** function to generate a **Binary Tree** in the following Zag-Zig shape. Write an insertion function which adds one node at a time in a tree and grows like the attached tree. E.g. in this case nodes were inserted in this sequence: **14, 12, 3, 9, ...** [7]



1

```
int count=0;

void ins (node * temp)
{ if(temp→left==NULL && temp→right==NULL)
  { if (count%2==0)
    { ~~temp→left= new node~~;
         n= new node;
           ~~cin>>data~~;
      ~~temp→left~~
         cin>>n→data;
      temp→lef= n; count++;
    }
    if (count%2!=0)
    { n=new node;  cin>>n→data;
      temp→~~left~~right = n; count++;
    }
    else{
      ~~ins~~ if(count%2!=0)
        { ins(temp→right);
        }
      else
        { ins(temp→left);
        }
    }
}
```

No Connection

4

0

0.5

0.5

0

~~Q~~ 4) Assume a **Binary Tree** already exists in memory. Write a function definition which finds whether all the leaves are at the same level or not. [7]

```
~~int max_left_(node *temp)~~
~~{if(temp->left == NULL)~~
void leave_at_same ()
{  if((temp->left!=NULL)&&
      (temp->right!=NULL))
  {
  int a = L-L (temp->left);
  int b = L-R(temp->left);
  int c = R-L (temp->right);
  int d = R-R(temp->right);
  if (a==b && a==c && b==c && a==d
        && b=d && c==f)
    { cout<<"Same level"; }
  else
    { cout<<"Not"; }
  }
}
int L-L(node * temp)
{   if(temp->left==NULL)
      { return L-L max;
      }
~~else~~
~~{temp else~~
~~else{ L-L (temp->left)~~
else { L-L max++;
return L-L (temp->left);
}}
```

```
int L-R (node * temp)
{ if(temp->~~fe~~ Right==NULL)
  {return L-Rmax;
  }
  else{ L-Rmax++;
     return LR (temp->right);
  }}
int R-R (node * temp)
{ if(temp->Right==NULL)
    { return R-R max;
    }
  else{ R-R max++;
      return R-R(temp->right);
  }
int R-L (node * temp)
  { if(temp->left==NULL)
      { return R-L max;
    }
  else { R-L max++;
      return R-L(temp->left);
```

Best of Luck!